

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC ĐỒNG THÁP**



**TIỂU LUẬN
MÔN HỌC: THỊ GIÁC MÁY TÍNH**

**ĐỀ TÀI
CHẤM CÔNG VÀ GIÁM SÁT AN NINH TỰ ĐỘNG
SỬ DỤNG MTCNN VÀ FACENET
CHUYÊN NGÀNH: KHOA HỌC MÁY TÍNH**

Người hướng dẫn: PGS.TS Đoàn Thanh Nghị

Học viên: Trần Phụng Tường Như, MSSV: 0022450036

Lớp: KHMTi-B1K12

Chuyên ngành: Khoa học Máy tính

Năm học: 2023-2025

Đồng Tháp, năm 2023

LỜI CẢM ƠN

Trân trọng cảm ơn giáo viên hướng dẫn PGS.TS. Đoàn Thanh Nghị, Khoa Công Nghệ Thông Tin , trường Đại học An Giang đã giảng dạy, cung cấp cho chúng tôi những kiến thức từ cơ bản đến nâng cao chuyên ngành Thị giác máy tính và tận tình hướng dẫn lớp trong suốt quá trình làm đề án môn học.

Xin cảm ơn tác giả các công trình nghiên cứu mà chúng tôi đã tham khảo và có trích dẫn trong nội dung đề tài, đã có những đóng góp quan trọng trong lĩnh vực nghiên cứu, từ đó chúng tôi kế thừa, ứng dụng để hoàn thành nghiên cứu này.

Sinh viên thực hiện đề tài

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN	1
1.1 Giới thiệu	1
1.1.1 Tính cấp thiết	2
1.1.2 Ý nghĩa khoa học, thực tiễn	3
1.1.3 Một số nghiên cứu liên quan.....	3
1.2 Mô tả tóm tắt hệ thống.....	4
1.2.1 Dữ liệu hệ thống	5
1.2.2 Chức năng hệ thống	5
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	7
2.1 Bài toán nhận dạng khuôn mặt:	7
2.2 Nhận dạng khuôn mặt bằng thuật toán MTCNN kết hợp với FaceNet	7
2.2.1 Giới thiệu thuật toán MTCNN	7
2.2.2 Giới thiệu thuật toán FaceNet	11
CHƯƠNG 3. KẾT QUẢ VÀ THẢO LUẬN.....	14
3.1 Cài đặt các công cụ lập trình triển khai bài toán:.....	14
3.1.1 Công cụ và công nghệ triển khai	14
3.1.2 Cấu trúc thư mục dự án chương trình	14
3.1.3 Thiết kế cơ sở dữ liệu của chương trình:	14
3.2 Mô tả hoạt động của của hệ thống:	17
3.2.1 Thu thập bộ ảnh huấn luyện:.....	18
3.2.2 Huấn luyện dữ liệu:.....	21
3.2.3 Hệ thống nhận dạng đối tượng:	23
3.3 Kết quả chạy chương trình.....	29
3.3.1 Tạo lập các bộ ảnh huấn luyện.....	29
3.3.2 Huấn luyện dữ liệu.....	30
3.3.3 Chạy chương trình giám sát, chấm công.....	31
3.4 Phân tích kết quả, thảo luận và những vấn đề chưa được giải quyết.....	32
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	34

DANH MỤC HÌNH ẢNH

Hình 2-1 Mô hình kiến trúc P-Net [6]	8
Hình 2-2 Mô hình kiến trúc R-Net [6]	9
Hình 2-3 Mô hình kiến trúc O-Net [6].....	9
Hình 3-1 Cấu trúc cơ sở dữ liệu của chương trình.....	15
Hình 3-2 Tạo dựng cơ sở dữ liệu trong SQLite3	17
Hình 3-3 Chương trình chụp ảnh huấn luyện	30
Hình 3-4 Chương trình huấn luyện dữ liệu khuôn mặt.....	30
Hình 3-5 Chương trình giám sát cảnh báo người lạ xâm nhập trái phép và chấm công.....	31
Hình 3-6 Thông tin cảnh báo và chấm công cùng hình ảnh được gửi tự động lên Group Chat Telegram.....	32

CHƯƠNG 1. TỔNG QUAN

1.1 Giới thiệu

Trí tuệ nhân tạo (AI) là một nhánh rộng lớn của khoa học máy tính tập trung vào khả năng của máy để tạo ra hành vi hợp lý từ các đầu vào bên ngoài. Mục tiêu của AI là tạo ra các hệ thống có thể thực hiện các tác vụ đòi hỏi trí thông minh của con người.

Thị giác máy tính (Computer Vision) là một lĩnh vực trong trí tuệ nhân tạo (AI) và khoa học máy tính, tập trung vào việc phát triển các thuật toán và hệ thống để máy tính có thể hiểu và diễn giải thông tin từ hình ảnh và video. Mục tiêu chính của thị giác máy tính là giúp máy tính "nhìn thấy" và hiểu được thế giới xung quanh, tương tự như cách mà con người nhìn và hiểu.

Thị giác máy tính hiện nay đang được ứng dụng nhiều trong thực tiễn như: Giúp hệ thống xe tự hành phát hiện và tránh vật cản trên đường, nhận diện và tuân thủ các biển báo giao thông, nhận diện làn đường để xe di chuyển đúng vị trí trong làn đường; Trong y tế, ứng dụng hỗ trợ phân tích ảnh chụp X quang, MRI, CT để phát hiện bệnh lý, giám sát tình trạng sức khỏe của bệnh nhân thông qua hình ảnh video; trong An ninh cho phép nhận diện khuôn mặt, giám sát an ninh, phát hiện người lạ, kiểm tra danh tính tại sân bay, tòa nhà công sở, nhận dạng biển số xe, phát hiện hành vi vi phạm giao thông; Trong nông nghiệp hỗ trợ giám sát phát hiện bệnh lý cây trồng, giám sát phát hiện sâu rầy có hại, phân loại đánh giá chất lượng sản phẩm nông nghiệp và nhiều lĩnh vực khác.

Tại VNPT Đồng Tháp, nhu cầu ứng dụng AI nói chung và thị giác máy tính nói riêng còn rất lớn. Một trong những nhu cầu cấp thiết đó là ứng dụng AI nhận dạng khuôn mặt để chấm công tự động thay thế cho hệ thống chấm công cũ và giám sát an ninh tại trụ sở chính và các nhà kho, trạm viễn thông của đơn vị nhằm phát hiện, cảnh báo người lạ xâm nhập trái phép.

1.1.1 Tính cấp thiết

Chấm công, chấm công là một nhu cầu của nhiều tổ chức, doanh nghiệp. Có nhiều hình thức chấm công, chấm công như: Bảng thẻ giấy, thẻ từ, vân tay, khuôn mặt, móng mắt, bằng định vị GPS... Mỗi phương pháp chấm công đều có những ưu nhược điểm riêng. VNPT Đồng Tháp trong thời gian qua cũng đã triển khai một số hình thức chấm công như: sử dụng thẻ, máy đọc vân tay... Qua thời gian sử dụng các hình thức chấm công dùng thẻ có nhiều nhược điểm như phải in thẻ cho nhân viên, nhân viên có thể quên mang theo thẻ chấm công, thẻ bị hỏng, gian lận trong chấm công khó kiểm soát... Hình thức chấm công vân tay có đặc điểm là tất cả nhân viên đều phải tiếp xúc với thiết bị đọc vân tay có thể không an toàn khi có dịch bệnh, thao tác quét vân tay có thể không thuận tiện, máy đọc vân tay có thể xác thực thất bại khi tay bị ướt, ra mồ hôi, tay quá khô, tay bị xây xước... do đó việc triển khai hệ thống chấm công tự động sử dụng nhận dạng khuôn mặt là cần thiết và phù hợp để khắc phục các nhược điểm của các hình thức chấm công trên.

Đối với giám sát an ninh kho bãi nhà trạm, hiện nay VNPT Đồng Tháp đang sử dụng các hệ thống Camera giám sát an ninh thông thường, chưa có hệ thống tự động nhận dạng cảnh báo người lạ xâm nhập. Việc theo dõi video bằng nhân công không hiệu quả trong giám sát an ninh. Khi có xảy ra thất thoát tài sản việc xem lại các video lưu trữ từ Camera An Ninh lắp đặt tại hiện trường (nếu có) và tua đi tua lại video theo phương pháp truyền thống để xác định thời điểm có xâm nhập mất nhiều thời gian, và trong nhiều trường hợp, đối tượng đã rời đi rất xa, rất khó xử lý thu hồi tài sản.

Chính vì những lý do trên việc ứng dụng Trí tuệ nhân tạo trong nhận dạng khuôn mặt để chấm công và giám sát an ninh phát hiện cảnh báo người lạ xâm nhập là hết sức cấp thiết nhằm nâng cao hiệu quả công việc và tăng cường an ninh bảo vệ tài sản, thiết bị cho VNPT Đồng Tháp.

1.1.2 Ý nghĩa khoa học, thực tiễn

- Ý nghĩa khoa học: Đề xuất ứng dụng thị giác máy tính nhận dạng khuôn mặt kết hợp việc chấm công và giám sát an ninh cho trụ sở làm việc và các nhà trạm, kho bãi.

- Ý nghĩa thực tiễn: Giải quyết được bài toán chấm công và giám sát an ninh; Nâng cao hiệu quả công việc và bảo vệ tài sản cho VNPT Đồng Tháp

1.1.3 Một số nghiên cứu liên quan

Năm 2004, Paul Viola và Michael Jones trong các bài báo của họ [1][2] đã đề xuất thuật toán sử dụng các đặc trưng Haar-Like và giải thuật tăng tốc AdaBoost để huấn luyện các bộ phân loại xếp tầng, đạt được hiệu suất nhận dạng khuôn mặt người tốt và hiệu quả theo thời gian thực. Tuy nhiên, một số công trình nghiên cứu khác [3][4] đã chỉ ra rằng giải thuật của Viola-Jones mang tính đột phá và được sử dụng rộng rãi, nhưng nó có những hạn chế trong việc xử lý phát hiện khuôn mặt từ nhiều góc nhìn khác nhau và các thay đổi về tư thế, điều kiện ánh sáng và việc khuôn mặt bị che khuất một phần.

Gần đây, giải thuật CNN (Convolutional Neural Network) được đề xuất bởi Yann LeCun và các cộng sự [5] đã có những bước tiến lớn trong thị giác máy tính. Kaipeng Zhang và cộng sự đã đề xuất thuật toán MTCNN (Multi-task Cascaded Convolutional Networks) [6], là một giải pháp kết hợp cả phát hiện khuôn mặt và căn chỉnh khuôn mặt trong cùng một mô hình, sử dụng ba mạng nơ-ron tích chập liên tiếp để cải thiện độ chính xác và hiệu quả của quá trình phát hiện và căn chỉnh khuôn mặt. Công trình này đã có ảnh hưởng lớn trong cộng đồng nghiên cứu và ứng dụng trong lĩnh vực nhận dạng khuôn mặt, trở thành một trong những nền tảng quan trọng cho các phương pháp tiếp theo sử dụng học sâu trong nhận dạng khuôn mặt.

Tiếp theo, một nhóm các nhà nghiên cứu từ Google gồm Florian Schroff, Dmitry Kalenichenko, và James Philbin đã đề xuất thuật toán FaceNet, là một phương pháp mới để nhận dạng và phân cụm khuôn mặt bằng cách học một không gian nhúng (embedding space), trong đó khoảng cách Euclid tương ứng

với sự tương đồng về khuôn mặt [7]. Phương pháp này đã đạt được hiệu suất rất cao trên các tập dữ liệu chuẩn và đã trở thành một trong những phương pháp tiên tiến nhất trong lĩnh vực nhận dạng khuôn mặt hiện nay.

Năm 2021, Jin Rongrong cùng các cộng sự đã đề xuất một khung ứng dụng dựa trên CNN xếp tầng đa tác vụ kết hợp với tính năng nhúng thống nhất FaceNet để phát hiện và nhận dạng khuôn mặt. Các kết quả thử nghiệm đã chứng minh rằng phương pháp này dựa trên các phương pháp tiên tiến vượt qua tiêu chuẩn AFLW (Annotated Facial Landmarks in the Wild) đầy thử thách để căn chỉnh khuôn mặt. Ba đóng góp chính cho việc cải thiện hiệu suất là kiến trúc CNN xếp tầng được khử ký hiệu một cách cẩn thận, chiến lược khai thác mẫu cứng trực tuyến và học tập căn chỉnh khuôn mặt chung [8].

1.2 Mô tả tóm tắt hệ thống

Đồ án đề xuất xây dựng hệ thống Camera giám sát an ninh tại các trụ sở làm việc, các nhà kho và trạm Viễn thông của VNPT Đồng Tháp, sử dụng ứng dụng nhận dạng khuôn mặt để chấm công nhân sự làm việc và đồng thời giám sát an ninh liên tục để phát hiện và cảnh báo các đối tượng là người lạ xâm nhập trái phép tại khu vực giám sát. Các camera được lắp đặt tại các vị trí ra vào của trụ sở nơi làm việc để thực hiện chấm công tự động và bên trong các nhà kho, trạm viễn thông để giám sát an ninh phát hiện cảnh báo người lạ xâm nhập trái phép. Khi đối tượng xuất hiện trong vùng quan sát, hệ thống tự động tiến hành phát hiện, nhận dạng khuôn mặt và định danh đối tượng.

Nếu khuôn mặt có trong dữ liệu huấn luyện, hệ thống sẽ trả về ID của người được nhận diện và thực hiện chấm công ghi nhận sự có mặt có nhân sự tại vị trí và thời điểm nhận diện.

Nếu khuôn mặt không có trong dữ liệu huấn luyện hoặc khuôn mặt có trong dữ liệu huấn luyện nhưng ID của khuôn mặt không được cấp quyền quản lý tại khu vực giám sát, hệ thống sẽ xem như đối tượng xâm nhập trái phép và tiến hành lưu ảnh bằng chứng, đồng thời gửi cảnh báo kèm theo ảnh của đối tượng lên Group Chat Telegram. Người quản lý kho, trạm (thành viên của

group chat telegram) sẽ lập tức có thông tin và ảnh đối tượng từ đó có hướng xử lý tức thì để bảo vệ tài sản và ổn định dịch vụ cung cấp cho khách hàng.

1.2.1 Dữ liệu hệ thống

Để phục vụ cho việc nhận dạng khuôn mặt kết hợp giám sát cảnh báo tự động và chấm công, hệ thống cần 2 cơ sở dữ liệu để thực hiện các chức năng của chương trình:

Cơ sở dữ liệu khuôn mặt:

Được sử dụng để nhận dạng và định danh khuôn mặt xuất hiện trong ảnh thu được từ camera. Trước khi tiến hành giám sát, hệ thống cần tạo trước cơ sở dữ liệu huấn luyện để có thể nhận dạng được khuôn mặt của những người được phép ra vào kho, trạm và ID tương ứng của người đó sử dụng trong CSDL khuôn mặt phải trùng khớp với ID trong cơ sở dữ liệu chương trình để có thể liên kết lấy thông tin và kiểm tra các quyền ra vào kho, trạm tại vị trí giám sát.

Cơ sở dữ liệu chương trình:

CSDL này lưu trữ thông tin nhân sự đang làm việc tại VNPT Đồng Tháp và dữ liệu cấp quyền quản lý của nhân sự đối với các vị trí lý kho, trạm nơi lắp đặt camera giám sát an ninh và dữ liệu phục vụ chấm công tự động. Khi hệ thống tiến hành nhận dạng khuôn mặt, trường hợp tìm được thông tin trùng khớp trong CSDL khuôn mặt sẽ trả về ID của khuôn mặt kèm theo khoảng cách chỉ ra độ trùng khớp của khuôn mặt so với dữ liệu huấn luyện. ID trả về được sử dụng truy vấn trong CSDL chương trình để lấy thông tin để chấm công hoặc để xác định nhân sự có được gán quyền ra vào tại các vị trí giám sát an ninh của đơn vị hay không từ đó có các xử lý cảnh báo xâm nhập trái phép phù hợp.

1.2.2 Chức năng hệ thống

Chức năng chụp ảnh từng nhân sự để hình thành bộ ảnh huấn luyện cho hệ thống nhận dạng khuôn mặt của chương trình.

Chức năng huấn luyện để tạo cơ sở dữ liệu khuôn mặt phục vụ cho quá trình nhận dạng và xác định danh tính khuôn mặt.

Chức năng giám sát thời gian thực để phát hiện, nhận diện, định danh khuôn mặt phục vụ cho việc chấm công và phát hiện người lạ xâm nhập trái phép tại các vị trí giá sát.

Chức năng chấm công ghi nhận danh tính người được chấm công, vị trí chấm công và thời gian chấm công tương ứng.

Chức năng gửi cảnh báo kèm theo hình ảnh đối tượng khi phát hiện có đối tượng là người lạ đang xâm nhập trái phép tại các vị trí giám sát lên Group Chat Telegram cho người quản lý nắm thông tin để có hướng xử lý kịp thời.

Chức năng tự động kết nối đến hệ thống phần mềm Telegram, sử dụng Chat Bot của Telegram để gửi tin nhắn và hình ảnh lên Group Chat thông qua các giao diện lập trình ứng dụng.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Bài toán nhận dạng khuôn mặt:

Nhận dạng khuôn mặt người (Face recognition) là một công nghệ có khả năng so khớp khuôn mặt người từ hình ảnh kỹ thuật số hoặc khung hình video với cơ sở dữ liệu khuôn mặt, từ đó có thể nhận dạng, xác định danh tính của khuôn mặt. Nhận dạng khuôn mặt là đối tượng nghiên cứu của ngành Thị giác máy tính (ComputerVision) và cũng là đối tượng nghiên cứu của ngành Sinh trắc học (Biometrics), tương tự như vân tay, móng mắt.

Trong những năm gần đây, chúng ta đã chứng kiến sự tiến bộ đáng kể của các công nghệ có thể phát hiện và nhận dạng khuôn mặt. Máy ảnh trên điện thoại di động hiện nay hầu hết được trang bị công nghệ này, khi sử dụng chúng ta có thể thấy một hình hộp xung quanh các khuôn mặt trong khung hình.

Hiện nay công nghệ nhận dạng khuôn mặt đã có những bước tiến lớn mang lại tốc độ nhận diện nhanh và độ chính xác cao, trong số các thuật toán nhận dạng và định danh khuôn mặt, đề tài chọn triển khai thuật toán MTCNN để phát hiện khuôn mặt và thuật toán FaceNet để định danh khuôn mặt trong cơ sở dữ liệu huấn luyện.

2.2 Nhận dạng khuôn mặt bằng thuật toán MTCNN kết hợp với FaceNet

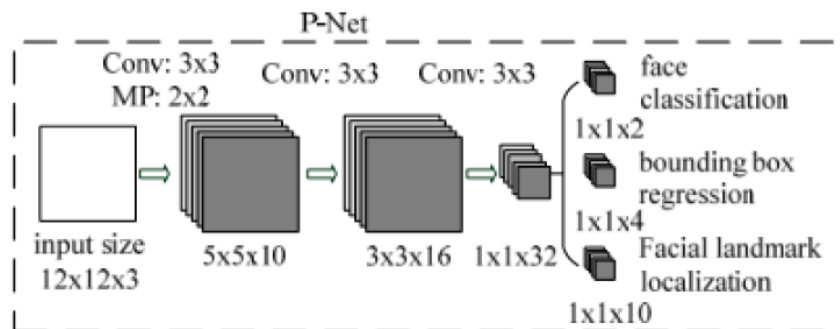
2.2.1 Giới thiệu thuật toán MTCNN

MTCNN (Multi-task Cascaded Convolutional Networks) là một thuật toán tiên tiến được thiết kế để phát hiện khuôn mặt và các điểm mốc trên khuôn mặt (facial landmarks) trong hình ảnh. Thuật toán này hoạt động dựa trên một cấu trúc gồm 3 mạng con ứng với 3 giai đoạn chính là P-Net, R-Net và O-Net, mỗi giai đoạn thực hiện một nhiệm vụ cụ thể và kết hợp để đạt được hiệu suất cao trong việc phát hiện khuôn mặt. Dưới đây là chi tiết về cơ sở lý thuyết của MTCNN.

2.2.1.1 Proposal Network (P-Net):

Một bức ảnh thông thường có thể có nhiều người xuất hiện với khoảng cách xa, gần khác nhau, do đó có thể xuất hiện nhiều khuôn mặt với kích cỡ to nhỏ khác nhau, trong cùng một bức ảnh. Nhiệm vụ của thuật toán là phát hiện toàn bộ các khuôn mặt cùng vị trí của các khuôn mặt đó trong bức ảnh. MTCNN sử dụng phương pháp định lại kích thước ảnh (resize) và nhân bản tạo ra một loại các ảnh từ to đến nhỏ như một kim tự tháp ảnh để có thể phát hiện khuôn mặt ở các kích thước khác nhau. Sau đó sử dụng một cửa sổ trượt (sliding windows) để quét qua hình ảnh, tạo ra các bounding boxes (hộp giới hạn), dự đoán tọa độ các bounding boxes trong ảnh, xác định xác suất mỗi vùng có thể chứa khuôn mặt, loại bỏ bớt các vùng chồng chéo để chọn ra các bounding boxes tốt nhất.

Ưu điểm của P-Net là có kiến trúc nhỏ gọn, có thể quét qua hình ảnh nhanh chóng, tạo ra các bounding box với độ chính xác sơ bộ cao, là nền tảng cho các mạng tiếp theo (R-Net và O-Net) để tinh chỉnh. Sơ đồ kiến trúc P-Net như hình 2.1.

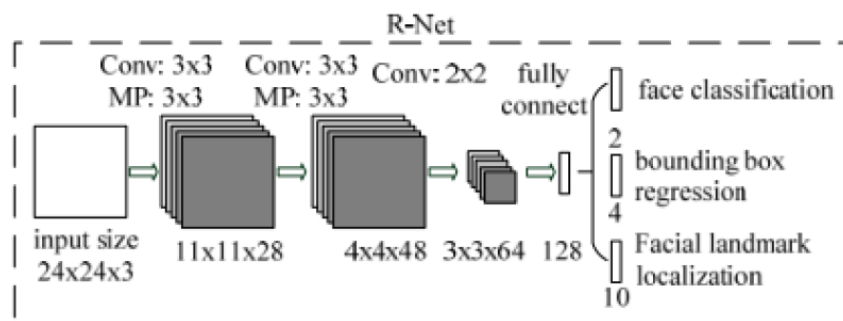


Hình 2.1 Mô hình kiến trúc P-Net [6]

2.2.1.2 Refinement Network (R-Net):

Refinement Network (R-Net) là thành phần thứ hai trong kiến trúc của MTCNN, có nhiệm vụ tinh chỉnh các bounding box được tạo ra từ Proposal Network (P-Net) và loại bỏ các bounding box không chính xác. R-Net có cấu trúc phức tạp hơn P-Net để đạt được độ chính xác cao hơn trong việc phát hiện khuôn mặt.

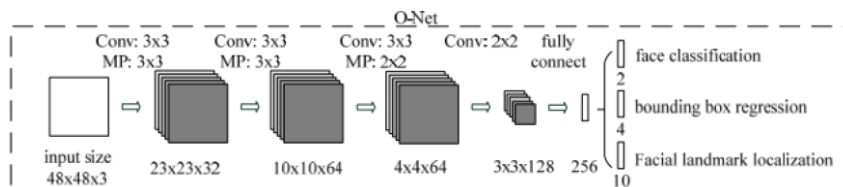
R-Net nhận đầu vào là các bounding box sơ bộ từ P-Net, sử dụng các lớp tích chập và các lớp kích hoạt trích xuất các đặc trưng chi tiết hơn từ các vùng ảnh này, dự đoán các tọa độ bounding box và điều chỉnh các vùng phát hiện sao cho chính xác hơn, xác định xác suất mỗi vùng chứa khuôn mặt, lọc qua NMS để loại bỏ các vùng chồng chéo và giữ lại các bounding box tốt nhất. R-Net xuất ra các bounding box đã được tinh chỉnh và các điểm mốc khuôn mặt (facial landmarks). Kiến trúc R-Net như hình 2.2



Hình 2.2 Mô hình kiến trúc R-Net [6]

2.2.1.3 Output Network (O-Net):

Output Network (O-Net) là thành phần cuối cùng trong kiến trúc của MTCNN. Nhiệm vụ của O-Net là tiếp tục tinh chỉnh các bounding box đã được R-Net lọc và xác định với độ chính xác cao nhất. Ngoài ra, O-Net còn dự đoán các điểm mốc khuôn mặt (facial landmarks) như mắt, mũi, và miệng. Mạng này xuất ra các bounding box chính xác cuối cùng, cùng với các điểm mốc khuôn mặt và giá trị confidence score. Kiến trúc O-Net như hình 2.3



Hình 2.3 Mô hình kiến trúc O-Net [6]

2.2.1.4 Multi-task Learning (Học đa nhiệm)

MTCNN sử dụng học đa nhiệm (multi-task learning) để cải thiện hiệu suất phát hiện khuôn mặt và điểm mốc khuôn mặt. Các nhiệm vụ chính bao gồm:

- **Bounding Box Regression:** Dự đoán vị trí và kích thước của bounding box.
- **Facial Landmark Localization:** Dự đoán vị trí của các điểm mốc khuôn mặt (mắt, mũi, miệng).
- **Classification:** Xác định xem một bounding box có chứa khuôn mặt hay không.

Mỗi mạng con trong MTCNN được huấn luyện để tối ưu hóa cho cả ba nhiệm vụ này, điều này giúp các mạng học được các đặc trưng đa dạng và tổng quát hơn.

2.2.1.5 Chiến lược Cascading

Chiến lược cascading (liên tiếp) của MTCNN giúp giảm thiểu số lượng false positives (dương tính giả) bằng cách lọc dần qua các bước mạng con:

- **P-Net:** Xác định các vùng có khả năng chứa khuôn mặt với độ tin cậy thấp hơn, nhưng ở nhiều tỷ lệ khác nhau.
- **R-Net:** Tinh chỉnh và loại bỏ các vùng sai lệch, cải thiện độ chính xác của bounding box.
- **O-Net:** Tinh chỉnh cuối cùng, xác định chính xác các bounding box và điểm mốc khuôn mặt.

2.2.1.6 Lợi ích của MTCNN

- **Độ chính xác cao:** Với chiến lược cascading và multi-task learning, MTCNN đạt được độ chính xác cao trong phát hiện khuôn mặt.
- **Tốc độ nhanh:** Mặc dù bao gồm ba giai đoạn, MTCNN vẫn có thể hoạt động nhanh chóng nhờ vào kiến trúc mạng nhẹ và tối ưu.
- **Khả năng xử lý đa tỷ lệ:** MTCNN có thể xử lý hình ảnh ở nhiều kích thước khác nhau, giúp phát hiện khuôn mặt ở các tỷ lệ và vị trí khác nhau trong hình ảnh.

Nhờ các đặc điểm trên, MTCNN trở thành một công cụ mạnh mẽ và phổ biến trong các ứng dụng liên quan đến phát hiện khuôn mặt, bao gồm giám sát an ninh, nhận diện khuôn mặt, và phân tích hình ảnh.

2.2.2 Giới thiệu thuật toán FaceNet

FaceNet là một mạng nơ-ron sâu được thiết kế bởi Google để giải quyết các vấn đề liên quan đến nhận dạng và xác thực khuôn mặt. Điểm mạnh của FaceNet nằm ở cách nó học biểu diễn khuôn mặt (face embeddings) dưới dạng vector trong không gian nhiều chiều, sao cho các khuôn mặt giống nhau gần nhau trong không gian vector, trong khi các khuôn mặt khác nhau thì cách xa nhau. Dưới đây là cách hoạt động chi tiết của FaceNet

2.2.2.1 Mục tiêu của FaceNet

Mục tiêu chính của FaceNet là ánh xạ (map) một hình ảnh khuôn mặt vào một vector có chiều dài cố định (thường là 128 chiều) sao cho:

Các vector của cùng một người phải gần nhau trong không gian vector.

Các vector của những người khác nhau phải cách xa nhau trong không gian vector.

2.2.2.2 Mô hình và Kiến trúc

FaceNet sử dụng một kiến trúc mạng nơ-ron tích chập sâu (deep convolutional neural network, CNN) để trích xuất các đặc trưng từ hình ảnh khuôn mặt. Mạng này có thể dựa trên các kiến trúc CNN phổ biến như Inception, ResNet, hoặc các kiến trúc CNN sâu khác.

2.2.2.3 Triplet Loss

Để học các đặc trưng khuôn mặt một cách hiệu quả, FaceNet sử dụng một hàm mất mát gọi là Triplet Loss. Triplet Loss giúp đảm bảo rằng các vector của cùng một người phải gần nhau và các vector của những người khác nhau phải cách xa nhau.

Hàm mất mát này được định nghĩa dựa trên ba hình ảnh:

- **Anchor (A)**: Hình ảnh khuôn mặt gốc.
- **Positive (P)**: Hình ảnh khuôn mặt của cùng một người với hình ảnh Anchor.

- **Negative (N):** Hình ảnh khuôn mặt của một người khác.

Triplet Loss được định nghĩa như sau:

$$L(A, P, N) = \max(0, \|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha)$$

Trong đó:

- $f(A)$, $f(P)$ và $f(N)$ là các embedding vector tương ứng của hình ảnh Anchor, Positive và Negative
- $\| \cdot \|$ là khoảng cách Euclidean
- α là một khoảng cách biên đôn (margin) để đảm bảo sự phân biệt giữa các vector.

Triple Loss đảm bảo rằng khoảng cách giữa embedding của Anchor và Negative ít nhất là α .

2.2.2.4 Quy trình Huấn luyện

Quy trình huấn luyện của FaceNet bao gồm các bước sau:

1. **Thu thập dữ liệu:** Thu thập bộ dữ liệu lớn gồm các hình ảnh khuôn mặt với nhãn xác định.
2. **Chuẩn bị triplets:** Tạo các bộ triplet (anchor, positive, negative) từ dữ liệu.
3. **Huấn luyện mạng:** Sử dụng các bộ triplet để huấn luyện mạng nơ-ron, tối ưu hóa hàm mất mát triplet loss.
4. **Tạo face embeddings:** Sử dụng mạng đã huấn luyện để tạo face embeddings cho các khuôn mặt.

2.2.2.5 Ứng dụng của FaceNet

Sau khi huấn luyện, FaceNet có thể được sử dụng cho nhiều ứng dụng liên quan đến nhận diện khuôn mặt:

- **Nhận dạng khuôn mặt:** Xác định danh tính của một người bằng cách so sánh face embedding của họ với một cơ sở dữ liệu đã biết.
- **Xác thực khuôn mặt:** Kiểm tra xem hai khuôn mặt có phải là cùng một người hay không bằng cách đo khoảng cách giữa các face embedding.
- **Tập trung vào ảnh nhóm:** Phân cụm các khuôn mặt giống nhau trong một ảnh nhóm.

CHƯƠNG 3. KẾT QUẢ VÀ THẢO LUẬN

3.1 Cài đặt các công cụ lập trình triển khai bài toán:

3.1.1 Công cụ và công nghệ triển khai

Đồ án sử dụng ngôn ngữ lập trình Python phiên bản 3.12.1, môi trường soạn thảo và quản lý mã nguồn Visual Studio code chạy môi trường Microsoft Windows 10, 64 bit để xây dựng ứng dụng.

Hệ quản trị cơ sở dữ liệu: SQLite3.

Công nghệ nhận dạng khuôn mặt được sử dụng: MTCNN và FaceNet

Phương pháp thiết lập kênh cảnh báo: Chatbot trên Telegram

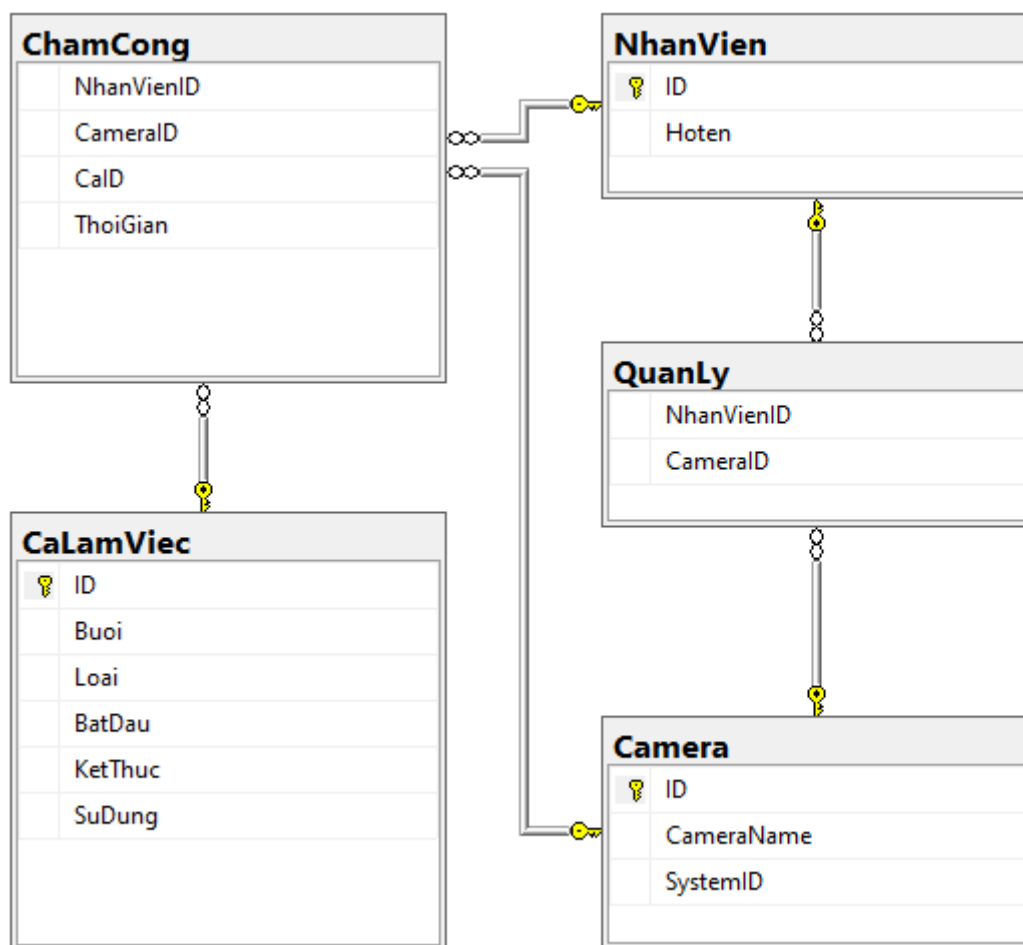
3.1.2 Cấu trúc thư mục dự án chương trình

Tên thư mục / tập tin	Mô tả
DiemDanhGiamSat	Thư mục dự án
Align	Thư mục chứa một số thư viện FaceNet
Dataset	Thư mục con chứa các bộ ảnh huấn luyện
Models	Thư mục con chứa các file dữ liệu huấn luyện
proofData	Thư mục con chứa ảnh chụp của đối tượng xâm nhập trái phép
Data.db	Cơ sở dữ liệu SQLite của chương trình
face_capture.py	Chương trình Python để chụp ảnh huấn luyện
face_training.py	Chương trình Python để huấn luyện dữ liệu
face_regconition.py	Chương trình Python để giám sát và chấm công

3.1.3 Thiết kế cơ sở dữ liệu của chương trình:

Để dễ dàng quản lý thông tin của hệ thống các camera, thông tin của người quản lý và thông tin về phân quyền cho phép ra vào các kho, trạm Viễn Thông, chương trình cần có một cơ sở dữ liệu để lưu trữ, truy vấn thông tin.

Để hệ thống gọn nhẹ, vừa đủ phục vụ giải quyết bài toán nhận diện khuôn mặt, phát hiện cảnh báo người lạ xâm nhập trái phép tại các kho và trạm Viễn thông, nhóm tác giả đề xuất cấu trúc cơ sở dữ liệu gồm có 5 bảng như hình 3.1.



Hình 3.1 Cấu trúc cơ sở dữ liệu của chương trình

1. Bảng Camera: Lưu trữ thông tin của các camera gồm:
 - ID: Kiểu số, là khoá chính
 - CameraName: Kiểu chuỗi, chứa tên Camera. Để thuận tiện trong việc dễ dàng xác định thông tin khu vực lắp đặt Camera
 - SystemID: Kiểu số, là ID khi kết nối Camera vào hệ thống máy tính chạy chương trình giám sát.
2. NhanVien: Lưu trữ thông tin nhân viên
 - ID: Kiểu số, khoá chính
 - Hoten: Kiểu chuỗi lưu họ tên
3. Bảng CaLamViec
 - ID: Kiểu số, là khoá chính
 - ID: Kiểu số, khoá chính
 - Buoi: Kiểu chuỗi lưu buổi làm việc như SANG, CHIEU

Loai: Kiểu text phân loại chấm công VAO hay RA

Batdau: Thời gian bắt đầu cho phép chấm công của ca làm việc này

Ketthuc: Thời gian kết thúc cho phép chấm công của ca làm việc này.

Sudung: Kiểu Int mẫu tin này đang sử dụng hoặc không sử dụng

4. Bảng QuanLy: Lưu trữ thông tin phân công quản lý tại vị trí Camera

NhanVienID: Kiểu số, khoá ngoại kết nối đến bảng NhanVien

CameraID: Kiểu số, khoá ngoại, kết nối bảng Camera

5. Bảng ChamCong: Lưu trữ thông tin chấm công

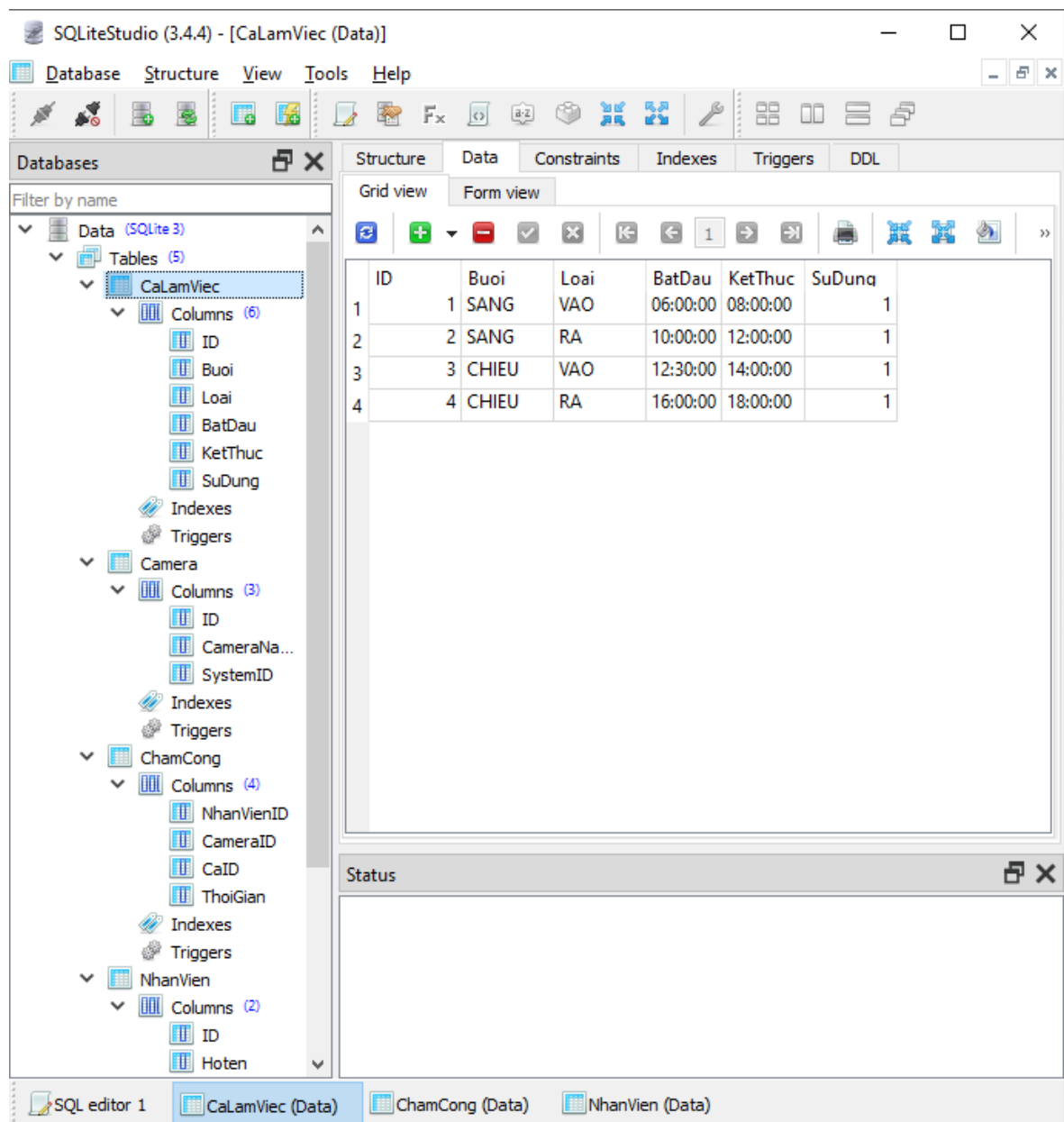
NhanVienID: ID của nhân viên

CameraID: ID của Camera tại nơi chấm công

CaID: ID của ca làm việc được chấm công

ThoiGian: Thời gian chấm công.

Cấu trúc dữ liệu được tạo trên hệ quản trị cơ sở dữ liệu SQLite3 như hình 3.2



Hình 3.2 Tạo dựng cơ sở dữ liệu trong SQLite3

3.2 Mô tả hoạt động của của hệ thống:

Hệ thống camera đặt tại các khu vực chấm công và giám sát (trụ sở làm việc, bên trong nhà kho hoặc trạm Viễn Thông). Sau khi lắp các camera tại các vị trí có góc máy phù hợp và điều kiện ánh sáng tốt nhất cho việc nhận dạng khuôn mặt. Khi đối tượng xuất hiện trong vùng quan sát, hệ thống tự động tiến hành phát hiện, định danh đối tượng để chấm công hoặc giám sát.

Trường hợp chấm công: Camera đặt tại khu vực làm việc và chương trình thiết lập Camera thực hiện chức năng chấm công. Khi có người xuất hiện trong

phạm vi nhận dạng, nếu khuôn mặt tìm thấy trong cơ sở dữ liệu chấm công, hệ thống ghi nhận chấm công thành công và ghi lại ID, họ tên, ngày giờ địa điểm chấm công. Nếu không xác định được danh tính khuôn mặt, chương trình bỏ qua.

Trường hợp giám sát an ninh: Camera lắp đặt tại các vị trí cần giám sát an ninh. Khi có người xuất hiện trong khung hình, hệ thống sẽ nhận dạng khuôn mặt và so khớp với dữ liệu huấn luyện. Nếu khuôn mặt có trong dữ liệu huấn luyện, hệ thống sẽ trả về ID của người được nhận diện và kiểm tra tiếp ID được gán quyền quản lý kho, trạm tại vị trí giám sát, hệ thống xem như sự xuất hiện của đối tượng là hợp lệ và xét tiếp các khuôn mặt khác.

Nếu khuôn mặt không có trong dữ liệu huấn luyện hoặc khuôn mặt có trong dữ liệu huấn luyện nhưng ID của khuôn mặt không được cấp quyền quản lý tại khu vực giám sát, hệ thống sẽ xem như đối tượng xâm nhập trái phép và tiến hành lưu ảnh bằng chứng, đồng thời gửi cảnh báo kèm theo ảnh của đối tượng lên Group Chat Telegram. Người quản lý kho, trạm (thành viên của group chat telegram) sẽ lập tức có thông tin và ảnh đối tượng từ đó có hướng xử lý tức thì để bảo vệ tài sản.

3.2.1 Thu thập bộ ảnh huấn luyện:

Đề tài chọn phương án chụp ảnh tự động bằng chương trình. Người cần chụp ảnh huấn luyện đứng trước Camera, và chạy chương trình face_capture.py, hệ thống tiến hành chụp khoảng 20 (có thể khai báo số lượng ảnh chụp ở biến maxSample) ảnh khuôn mặt lưu vào thư mục Dataset/ID-cần nhận dạng khuôn mặt,

Mã lệnh Python của chương trình: face_capture.py

```
import tensorflow as tf
from imutils.video import VideoStream
import facenet
import imutils
import os
import align.detect_face
import numpy as np
import cv2
import collections
```

```

from sklearn.svm import SVC
import os

#Nhập ID nhân viên để chụp ảnh tự động làm dữ liệu nhận dạng
ID = input("Nhập ID: ")
user_path = 'DataSET/' + ID
# Tạo thư mục con (bao gồm các thư mục cha nếu chúng chưa tồn tại)
os.makedirs(user_path, exist_ok=True)

# set text style
fontface = cv2.FONT_HERSHEY_SIMPLEX
fontscale = 4
fontcolor = (203, 23, 252)
#Number of picture to capture
maxSample=20
sampleNum = 0

#MTCNN init
MIN_SIZE = 20
THRESHOLD = [0.6, 0.7, 0.7]
FACTOR = 0.709
IMAGE_SIZE = 182
INPUT_IMAGE_SIZE = 160
FACENET_MODEL_PATH = 'Models/20180402-114759.pb'

tf.compat.v1.disable_eager_execution()

with tf.Graph().as_default():
    #with tf.compat.v1.Session() as sess:

        # Cài đặt GPU nếu có
        gpu_options =
tf.compat.v1.GPUOptions(per_process_gpu_memory_fraction=0.6)
        sess =
tf.compat.v1.Session(config=tf.compat.v1.ConfigProto(gpu_options=gpu_options, log_device_placement=False))
        with sess.as_default():
            # Load the model
            print('Loading feature extraction model')
            facenet.load_model(FACENET_MODEL_PATH)
            # Get input and output tensors
            images_placeholder =
tf.compat.v1.get_default_graph().get_tensor_by_name("input:0")
            embeddings =
tf.compat.v1.get_default_graph().get_tensor_by_name("embeddings:0")
            phase_train_placeholder =
tf.compat.v1.get_default_graph().get_tensor_by_name("phase_train:0")
            embedding_size = embeddings.get_shape()[1]
            pnet, rnet, onet = align.detect_face.create_mtcnn(sess, "align")
            people_detected = set()

```

```

person_detected = collections.Counter()
cap = VideoStream(src=0).start()
while (True):
    frame = cap.read()
    frame = imutils.resize(frame, width=600)
    frame = cv2.flip(frame, 1)
    bounding_boxes, _ = align.detect_face.detect_face(frame,
MINSIZE, pnet, rnet, onet, THRESHOLD, FACTOR)
    faces_found = bounding_boxes.shape[0]

    if faces_found > 0:
        det = bounding_boxes[:, 0:4]
        for i in range(faces_found):
            x = np.int32(det[i][0])
            y = np.int32(det[i][1])
            x2 = np.int32(det[i][2])
            y2 = np.int32(det[i][3])
            cropped = frame[y:y2, x:x2, :]
            scaled = cv2.resize(cropped, (INPUT_IMAGE_SIZE,
INPUT_IMAGE_SIZE),
                                interpolation=cv2.INTER_CUBIC)
            # incrementing sample number
            sampleNum = sampleNum + 1
            # saving the captured face in the dataset folder
            #Tang do sang cho hinh nanh
            #scaled = cv2.convertScaleAbs(scaled, alpha=1,
beta=20)

            #Luu anh vao thu muc ID nhan vien trong thu muc
DataSET

            cv2.imwrite(user_path + '/' + str(sampleNum) + ".jpg",
scaled)

            #Vẽ hình chu nhật bao quanh khuôn mặt, in text bên
duoi

            cv2.rectangle(frame, (x, y), (x2, y2), (255, 0, 0), 2)
            cv2.putText(frame, str(sampleNum), (x, y2 + 100),
fontface, fontscale, fontcolor, 2)
            cv2.imshow('Capturing', frame)

            # wait for 300 milliseconds
            if cv2.waitKey(500) == ord('q'):
                break
            # break if the sample number is morethan maxSample
            if sampleNum >= maxSample:
                break
cap.release()
cv2.destroyAllWindows()

```


3.2.2 Huấn luyện dữ liệu:

Sau khi đã chụp đủ các bộ ảnh gương mặt của những người cần nhận dạng khuôn mặt, ta tiến hành chạy chương trình huấn luyện dữ liệu để hình thành cơ sở dữ liệu khuôn mặt cho chương trình.

Đầu vào: Các bộ file ảnh chụp gương mặt lưu trong thư mục Dataset/ID định danh khuôn mặt

Đầu ra: File facemodel.pkl, lưu ở thư mục Models chứa dữ liệu huấn luyện của toàn bộ các khuôn mặt từ các file ảnh trong thư mục Dataset.

Mã lệnh Python của chương trình: face_training.py

```
import tensorflow as tf
import numpy as np
import facenet
import os
import math
import pickle
from sklearn.svm import SVC

def split_dataset(dataset, min_nrof_images_per_class,
nrof_train_images_per_class):
    train_set = []
    test_set = []
    for cls in dataset:
        paths = cls.image_paths
        # Remove classes with less than min_nrof_images_per_class
        if len(paths) >= min_nrof_images_per_class:
            np.random.shuffle(paths)
            train_set.append(facenet.ImageClass(cls.name,
paths[:nrof_train_images_per_class]))
            test_set.append(facenet.ImageClass(cls.name,
paths[nrof_train_images_per_class:]))
    return train_set, test_set

INPUT_IMAGE_SIZE = 160
BATCH_SIZE = 1000
DATASET_PATH = 'Dataset'
CLASSIFIER_PATH = 'Models/facemodel.pkl'
FACENET_MODEL_PATH = 'Models/20180402-114759.pb'
with tf.Graph().as_default():
    with tf.compat.v1.Session() as sess:
        #
        dataset = facenet.get_dataset(DATASET_PATH)

        # Check that there are at least one training image per class
```

```

        for cls in dataset:
            assert(len(cls.image_paths)>0, 'There must be at least one
image for each class in the dataset')

    paths, labels = facenet.get_image_paths_and_labels(dataset)

    print('Number of classes: %d' % len(dataset))
    print('Number of images: %d' % len(paths))

    # Load the model
    print('Loading feature extraction model')
    facenet.load_model(FACENET_MODEL_PATH)

    # Get input and output tensors
    images_placeholder =
tf.compat.v1.get_default_graph().get_tensor_by_name("input:0")
    embeddings =
tf.compat.v1.get_default_graph().get_tensor_by_name("embeddings:0")
    phase_train_placeholder =
tf.compat.v1.get_default_graph().get_tensor_by_name("phase_train:0")
    embedding_size = embeddings.get_shape()[1]

    # Run forward pass to calculate embeddings
    print('Calculating features for images')
    nrof_images = len(paths)
    nrof_batches_per_epoch = int(math.ceil(1.0*nrof_images /
BATCH_SIZE))
    emb_array = np.zeros((nrof_images, embedding_size))
    for i in range(nrof_batches_per_epoch):
        start_index = i*BATCH_SIZE
        end_index = min((i+1)*BATCH_SIZE, nrof_images)
        paths_batch = paths[start_index:end_index]
        images = facenet.load_data(paths_batch, False, False,
INPUT_IMAGE_SIZE)
        feed_dict = { images_placeholder:images,
phase_train_placeholder:False }
        emb_array[start_index:end_index,:] = sess.run(embeddings,
feed_dict=feed_dict)

    classifier_filename_exp = os.path.expanduser(CLASSIFIER_PATH)

    # Train classifier
    print('Training classifier')
    model = SVC(kernel='linear', probability=True)
    model.fit(emb_array, labels)

    # Create a list of class names
    class_names = [ cls.name.replace('_', ' ') for cls in dataset]

    # Saving classifier model

```

```

with open(classifier_filename_exp, 'wb') as outfile:
    pickle.dump((model, class_names), outfile)
    print('Saved classifier model to file "%s"' %
classifier_filename_exp)

```

3.2.3 Hệ thống nhận dạng đối tượng:

Sau khi hoàn tất việc huấn luyện dữ liệu, hệ thống đã sẵn sàng để chạy chương trình chấm công và giám sát, cảnh báo người lạ xâm nhập dựa trên thư viện MTCNN và FaceNet để phát hiện và định danh khuôn mặt.

Khi đối tượng xuất hiện trong vùng quan sát, hệ thống tự động tiến hành phát hiện, nhận dạng và định danh đối tượng.

Nếu khuôn mặt có trong dữ liệu huấn luyện, hệ thống sẽ trả về ID của người được nhận diện, nếu thời gian hiện tại nằm trong khung thời gian chấm công, chương trình kiểm tra tiếp nếu ID này chưa thực hiện chấm công thì hệ thống tự động thực hiện chấm công tại vị trí camera đồng thời gửi thông tin và hình ảnh lên Telegram xác nhận việc chấm công. Chấm công vào và ra chỉ thực hiện được đúng 1 lần trong ca làm việc đó.

Nếu khuôn mặt không được nhận diện hoặc khuôn mặt được nhận diện ID khuôn mặt nhưng tại vị trí camera đó ID nhân viên không có quyền xâm nhập, hệ thống xem như xâm nhập trái phép và lưu ảnh vào thư mục proofData và phát cảnh báo cùng hình ảnh lên kênh Telegram.

Mã lệnh Python của chương trình: face_regconition.py

```

import tensorflow as tf
from imutils.video import VideoStream
import facenet
import imutils
import pickle
import align.detect_face
import numpy as np
import cv2
import collections
from sklearn.svm import SVC
import sqlite3
import datetime
import requests
#=====

```

```

def layThongTinCamera(id):
    conn = sqlite3.connect("Data.db")
    cmd = "SELECT ID, CameraName, SystemID FROM Camera WHERE ID=" +
str(id)
    cursor = conn.execute(cmd)
    profile = None
    for row in cursor:
        profile = row
    conn.close()
    return profile

#Truy vấn dữ liệu người quản lý bởi ID
def layThongTinNhanVien(id):
    conn = sqlite3.connect("Data.db")
    cmd = "SELECT * FROM NhanVien WHERE ID=" + str(id)
    cursor = conn.execute(cmd)
    profile = None
    for row in cursor:
        profile = row
    conn.close()
    return profile

#Kiểm tra quyền Quan lý tại khu vực camera đang xét
def isAllowed(NhanVienID, CameraID):
    conn = sqlite3.connect("Data.db")
    cmd="Select * from NhanVien inner join QuanLy on
NhanVien.ID=QuanLy.NhanVienID " + \
        "where NhanVien.ID=" + str(NhanVienID) + " and QuanLy.CameraID=" +
str(CameraID)
    cursor = conn.execute(cmd)
    profile = None
    for row in cursor:
        profile = row
    if (profile == None):
        conn.close()
        return False
    else:
        conn.close()
        return True

#Hàm gửi tin nhắn và hình ảnh lên Group Telegram
def xuatThongBaoTelegram(cv2, img, TenCam, msg, CHAT_ID):
    stringDateTime = datetime.datetime.now().strftime("%Y-%m-%d-%a-%H-%M-%S")
    cv2.imwrite("proofData/" + TenCam + '-' + stringDateTime + ".jpg",
img)
    print("proofData/" + TenCam + '-' + stringDateTime + ".jpg")
    #Gửi cảnh báo text qua Group Telegram

```

```

    resp =
requests.post('https://api.telegram.org/bot<Token>/sendMessage?chat_id=-'
+ str(CHAT_ID)+ '&text=' + msg)
    #Gửi ảnh bằng chứng qua Group Telegram
    files = {'photo': open("proofData/" + TenCam + '-' + stringDateTime +
".jpg", 'rb')}
    resp =
requests.post('https://api.telegram.org/bot<Token>/sendPhoto?chat_id=-' +
str(CHAT_ID), files=files)

#Lấy ID chấm công của ca làm việc theo thời gian hiện tại
def layIDCaLamViec():
    conn = sqlite3.connect("Data.db")
    cmd="Select ID, Loai from CaLamViec " + \
    "WHERE time('now','localtime') BETWEEN BatDau and Ketthuc"
    cursor = conn.execute(cmd)
    conn.commit()
    rows = cursor.fetchall()
    if rows:
        CaID = rows[0][0]
        Loai = rows[0][1]
        conn.close()
        return CaID, Loai
    else:
        return 0,0

#Kiểm tra nếu đã chấm công cho ca làm việc này rồi
def daChamCong(NhanVienID, CaID):
    conn = sqlite3.connect("Data.db")
    cmd="SELECT * FROM ChamCong " + \
    "WHERE NhanVienID=" + str(NhanVienID) + " and date(ThoiGian) =
date('now','localtime')" + " and CaID=" + str(CaID)
    cursor = conn.execute(cmd)
    conn.commit()
    rows = cursor.fetchall()
    if rows:
        conn.close()
        return 1
    else:
        conn.close()
        return 0

#Kiểm tra và thực hiện chấm công
def KiemTraVaChamCong(NhanVienID, CameraID, CaID, Loai, cv2, frame,
CameraName):
    #Đang trong thời gian chấm công
    #Nếu đã chấm công RA thì cho phép cập nhật lại giờ chấm công
    if daChamCong(NhanVienID, CaID):
        if Loai == "RA": #Nếu đã chấm công ra thì cập nhật giờ chấm công
            conn = sqlite3.connect("Data.db")

```

```

        cursor = conn.cursor()
        cursor.execute("UPDATE ChamCong " + \
            "SET ThoiGian = datetime('now')"\
            "WHERE NhanVien.ID=" + str(NhanVienID) + " and date(ThoiGian)\
= date('now')" + " and CaID=" + str(CaID))
        conn.commit()
        conn.close()
    else:
        #Nếu chưa chấp công, thực hiện chấm công
        conn = sqlite3.connect("Data.db")
        cursor = conn.cursor()
        cursor.execute("INSERT INTO ChamCong (NhanVienID, CameraID, CaID,\
ThoiGian) VALUES (" + \
            "" + str(NhanVienID) + " , " + str(CameraID) + " , " + str(CaID) + \
            " , datetime('now', 'localtime'))")
        conn.commit()
        conn.close()
        xuatThongBaoTelegram(cv2, frame, CameraName, "Đã chấm công " + \
Loai, CHAMCONG_CHATID)
        print("Đã chấm công")

#Khởi tạo thông số hệ thống cho Camera
CameraID = 1
CameraProfile = layThongTinCamera(CameraID)
CameraName = CameraProfile[1]
SystemID = CameraProfile[2]
Interval = 1
#Khởi tạo ID Chat Group Telegram
CANHBAO_CHATID = <CHAT_ID CỦA GROUP CẢNH BÁO>
CHAMCONG_CHATID = <CHAT_ID CỦA GROUP CHẤM CÔNG>

#Khởi tạo thời gian giữa 2 lần cảnh báo
alertTime=datetime.datetime.now()- datetime.timedelta(minutes=Interval)
alertTime2=datetime.datetime.now()- datetime.timedelta(minutes=Interval)
#=====
MATCH_RATE = 0.7
MINSIZE = 20
THRESHOLD = [0.6, 0.7, 0.7]
FACTOR = 0.709
IMAGE_SIZE = 182
INPUT_IMAGE_SIZE = 160
CLASSIFIER_PATH = 'Models/facemodel.pkl'
FACENET_MODEL_PATH = 'Models/20180402-114759.pb'

# Enable TensorFlow 1.x compatibility mode
tf.compat.v1.disable_eager_execution()

# Load The Custom Classifier
with open(CLASSIFIER_PATH, 'rb') as file:
    model, class_names = pickle.load(file)

```

```

print("Custom Classifier, Successfully loaded")

with tf.Graph().as_default():
    # Cài đặt GPU neu co
    gpu_options =
tf.compat.v1.GPUOptions(per_process_gpu_memory_fraction=0.6)
    sess =
tf.compat.v1.Session(config=tf.compat.v1.ConfigProto(gpu_options=gpu_options, log_device_placement=False))
    with sess.as_default():
        # Load the model
        print('Loading feature extraction model')
        facenet.load_model(FACENET_MODEL_PATH)
        # Get input and output tensors
        images_placeholder =
tf.compat.v1.get_default_graph().get_tensor_by_name("input:0")
        embeddings =
tf.compat.v1.get_default_graph().get_tensor_by_name("embeddings:0")
        phase_train_placeholder =
tf.compat.v1.get_default_graph().get_tensor_by_name("phase_train:0")
        embedding_size = embeddings.get_shape()[1]
        pnet, rnet, onet = align.detect_face.create_mtcnn(sess, "align")
        people_detected = set()
        person_detected = collections.Counter()
        cap = VideoStream(src=0).start()
        while (True):
            frame = cap.read()
            frame = imutils.resize(frame, width=600)
            frame = cv2.flip(frame, 1)
            bounding_boxes, _ = align.detect_face.detect_face(frame,
MINSIZE, pnet, rnet, onet, THRESHOLD, FACTOR)
            faces_found = bounding_boxes.shape[0]
            try:
                if faces_found > 0:
                    det = bounding_boxes[:, 0:4]
                    bb = np.zeros((faces_found, 4), dtype=np.int32)
                    #Vẽ hình chữ nhật bao quanh các khuôn mặt được phát
hiện

                    for i in range(faces_found):
                        x = np.int32(det[i][0])
                        y = np.int32(det[i][1])
                        x2 = np.int32(det[i][2])
                        y2 = np.int32(det[i][3])
                        cv2.rectangle(frame, (x, y), (x2, y2), (0, 255,
0), 2)

                        #Xét từng khuôn mặt
                        for i in range(faces_found):
                            x = np.int32(det[i][0])
                            y = np.int32(det[i][1])
                            x2 = np.int32(det[i][2])

```

```

        y2 = np.int32(det[i][3])
        text_x = x
        text_y = y2 + 20
        #Kiểm tra tỷ lệ độ cao khuôn mặt so với khung hình
> 1 ngưỡng mong muốn
        if (y2-y)/frame.shape[0]>0.2:
            cropped = frame[y:y2, x:x2, :]
            scaled = cv2.resize(cropped,
(INPUT_IMAGE_SIZE, INPUT_IMAGE_SIZE),
                                interpolation=cv2.INTER_CU
BIC)
            scaled = facenet.prewhiten(scaled)
            scaled_reshape = scaled.reshape(-1,
INPUT_IMAGE_SIZE, INPUT_IMAGE_SIZE, 3)
            feed_dict = {images_placeholder:
scaled_reshape, phase_train_placeholder: False}
            emb_array = sess.run(embeddings,
feed_dict=feed_dict)

            predictions = model.predict_proba(emb_array)
            best_class_indices = np.argmax(predictions,
axis=1)

            best_class_probabilities = predictions[
np.arange(len(best_class_indices)),
best_class_indices]

            best_name = class_names[best_class_indices[0]]
            #Kiểm tra khoảng cách phân lớp đạt một giá trị
ngưỡng mong muốn
            if best_class_probabilities > MATCH_RATE:
                NhanVienID =
class_names[best_class_indices[0]]
                NhanVien = layThongTinNhanVien(NhanVienID)
                cv2.putText(frame, NhanVien[1], (text_x,
text_y), cv2.FONT_HERSHEY_COMPLEX_SMALL,
                                1, (255, 255, 255),
thickness=1, lineType=2)
                cv2.putText(frame,
str(round(best_class_probabilities[0], 3)), (text_x, text_y + 17),
                                cv2.FONT_HERSHEY_COMPLEX_SMALL
,
                                1, (255, 255, 255),
thickness=1, lineType=2)

                person_detected[best_name] += 1
                #Kiểm tra thực hiện chấm công
                #Nếu không phải giờ chấm công thì thoát ra
không xét

                CaID, Loai = layIDCaLamViec()
                if CaID != 0:
                    KiemTraVaChamCong(NhanVienID,
CameraID, CaID, Loai, cv2, frame, CameraName)

```



```

#Kiểm tra để cảnh báo xâm nhập
if not isAllowed(id, CameraID):
    diff = datetime.datetime.now() -
alertTime
    if (diff.seconds) >= Interval:
        msg = "Cảnh báo: Xâm nhập trái
phép - " + CameraName
        xuấtThôngBaoTelegram(cv2, frame,
CameraName, msg, CANHBAO_CHATID)
        alertTime =
datetime.datetime.now()
    else:
        name = "Unknown"
        cv2.putText(frame, name, (text_x, text_y),
cv2.FONT_HERSHEY_COMPLEX_SMALL,
1, (255, 255, 255),
thickness=1, lineType=2)
        diff = datetime.datetime.now() -
alertTime2
        if (diff.seconds) >= Interval:
            msg = "Cảnh báo: người lạ xâm nhập - "
+ CameraName
            xuấtThôngBaoTelegram(cv2, frame,
CameraName, msg, CANHBAO_CHATID)
            alertTime2 = datetime.datetime.now()
        except:
            pass

cv2.imshow('Camera:' + CameraName, frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

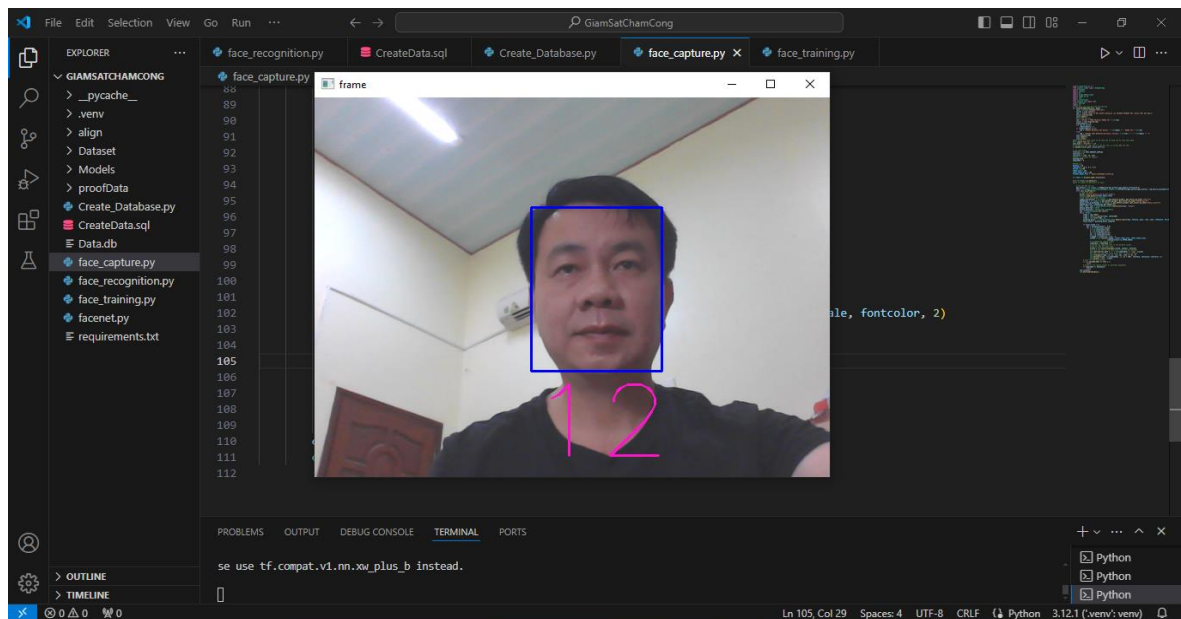
cap.release()
cv2.destroyAllWindows()

```

3.3 Kết quả chạy chương trình

3.3.1 Tạo lập các bộ ảnh huấn luyện

Hình 3.3 thể hiện kết quả chạy chương trình chụp hình tạo bộ ảnh huấn luyện của nhân viên.



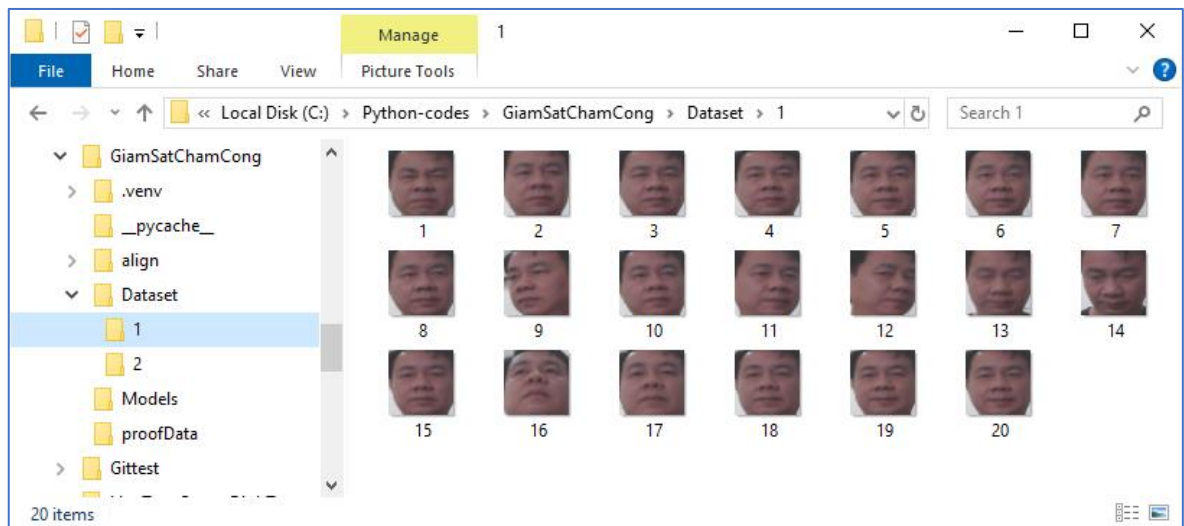
Hình 3.3 Chương trình chụp ảnh huấn luyện

3.3.2 Huấn luyện dữ liệu

Sau khi xong bước chụp ảnh huấn luyện. Tiến hành chạy chương trình face_training.py để huấn luyện dữ liệu khuôn mặt.

Đầu vào: Các file ảnh trong thư mục Dataset như Hình 3.4

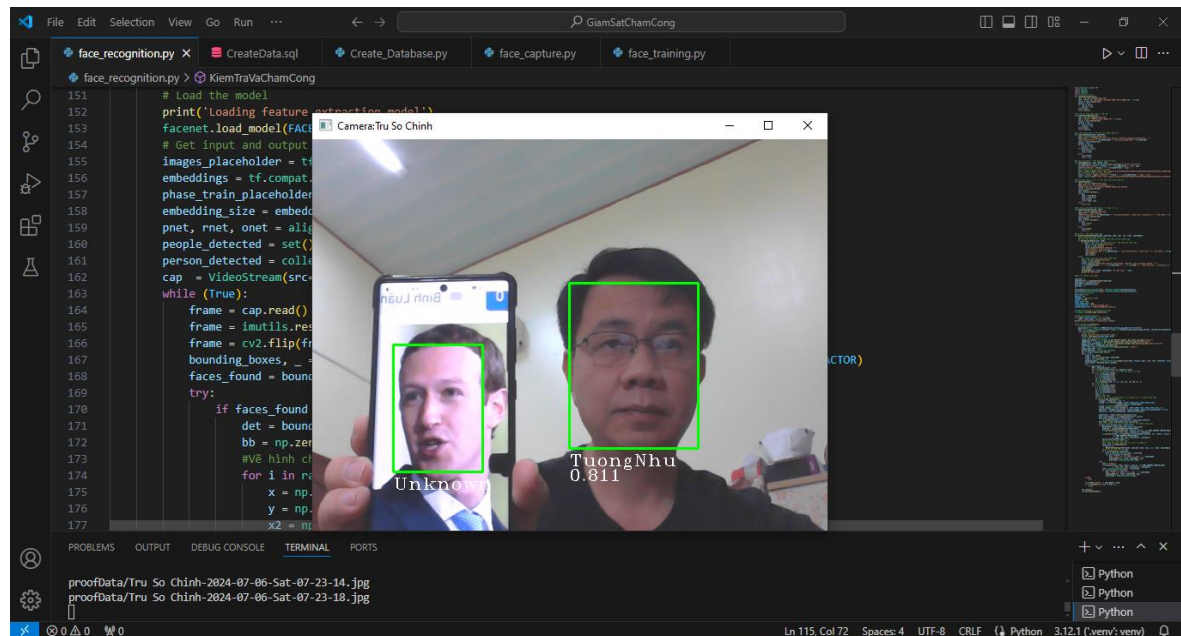
Đầu ra: File facemodel.pkl, lưu ở thư mục Models chứa dữ liệu huấn luyện của toàn bộ các khuôn mặt từ các file ảnh trong thư mục Dataset.



Hình 3.4 Chương trình huấn luyện dữ liệu khuôn mặt

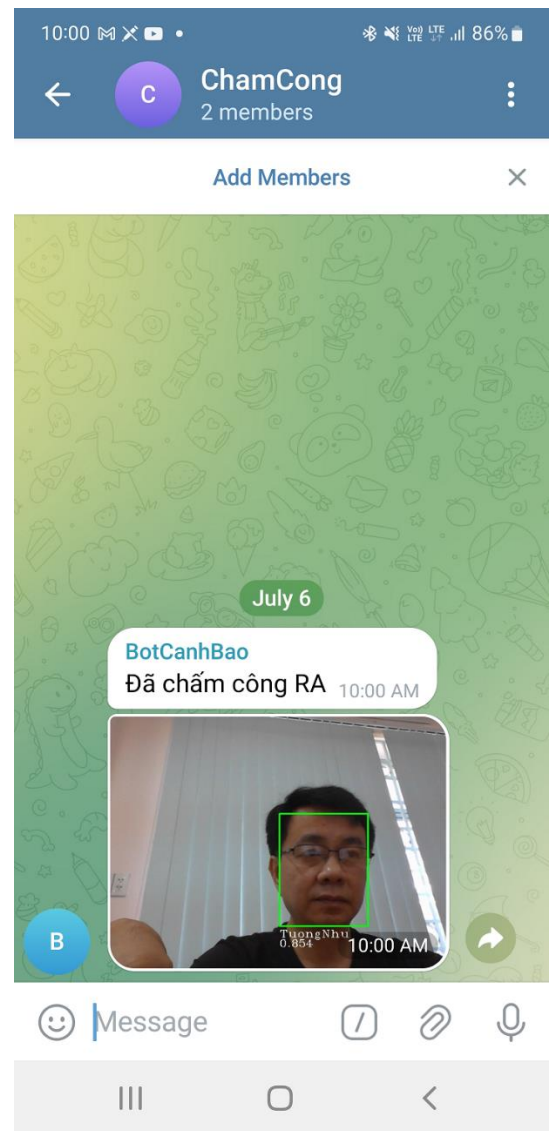
3.3.3 Chạy chương trình giám sát, chấm công

Chạy chương trình face_recognition.py, sau khi khởi tạo chương trình sẽ vào vòng lặp vô tận (ấn phím q để thoát) để đọc các frame ảnh từ camera theo thời gian thực và tiến hành nhận dạng, giám sát và chấm công tự động.



Hình 3.5 Chương trình giám sát cảnh báo người lạ xâm nhập trái phép và chấm công

Khi phát hiện đối tượng xâm nhập trái phép, chương trình sẽ lưu lại hình ảnh vào thư mục proofData, gửi tin nhắn cảnh báo cùng hình ảnh lên Group CanhBaoXamNhap trên Telegram. Khi nhận dạng được ID khuôn mặt, nếu trong thời hiện tại đang trong thời gian chấm công của ca làm việc chương trình sẽ thực hiện chấm công, lưu lại thông tin chấm công đồng thời gửi thông báo lên Group ChamCong cùng hình ảnh chấm công để người được chấm công có thể theo dõi kết quả chấm công của mình. Giao diện giám sát của chương trình thể như hình 3.5. Các group Telegram tiếp nhận thông báo và cảnh báo như hình 3.6. Kết quả chấm công như hình 3.7.



Hình 3.6 Thông tin cảnh báo và chấm công cùng hình ảnh được gửi tự động lên Group Chat Telegram

SQLiteStudio (3.4.4) - [BangChamCong (Data)]

Database Structure View Tools Help

Databases

Filter by name

- Data (SQLite 3)
 - Tables (5)
 - CaLamViec
 - Camera
 - ChamCong
 - NhanVien
 - QuanLy
 - Views (1)
 - BangChamCong

Query Data Triggers DDL

Grid view Form view

Filter data Total rows loaded: 6

	id	hoten	cameraname	buoi	loai	thoigian
1	1	TuongNhu	Tru So Chinh	SANG	VAO	2024-07-05 07:15:00
2	1	TuongNhu	Tru So Chinh	SANG	RA	2024-07-05 11:30:00
3	1	TuongNhu	Tru So Chinh	CHIEU	VAO	2024-07-05 13:00:00
4	1	TuongNhu	Tru So Chinh	CHIEU	RA	2024-07-05 17:00:00
5	1	TuongNhu	Tru So Chinh	SANG	VAO	2024-07-06 07:23:02
6	1	TuongNhu	Tru So Chinh	SANG	RA	2024-07-06 10:00:00

SQL editor 1 CaLamViec (Data) ChamCong (Data) NhanVien (Data) BangChamCong (Data)

Hình 3.7 Bảng chấm công

3.4 Phân tích kết quả, thảo luận và những vấn đề chưa được giải quyết

Qua kết quả thực nghiệm ở phần cho thấy bài toán đã được giải quyết theo yêu cầu ban đầu đặt ra. Hệ thống đã vận hành đúng theo thiết kế, nhận dạng định danh được các khuôn mặt có trong dữ liệu huấn luyện phục vụ cho chấm công và giám sát cảnh báo người lạ xâm nhập trái phép.

Do thời gian hạn hẹp nên đề tài mới chỉ triển khai để chương trình vận hành được theo yêu cầu, chưa xây dựng các giao diện dạng form.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Qua kết quả thực nghiệm ở phần cho thấy bài toán đã được giải quyết theo yêu cầu ban đầu đặt ra. Hệ thống đã vận hành đúng theo thiết kế, nhận dạng định danh được các khuôn mặt có trong dữ liệu huấn luyện phục vụ cho việc giám sát cảnh bảo người lạ xâm nhập trái phép và chấm công của nhân viên nơi làm việc.

2. Những hạn chế

Hạn chế của đề án là mới chỉ dừng lại ở qui mô nhỏ, trong quá trình thực nghiệm độ chính xác trong nhận dạng chưa cao, hệ thống chương trình được viết chưa đầy đủ các tiện ích để ứng dụng vào thực tiễn.

3. Hướng phát triển

Để hệ thống có thể ứng dụng được vào thực tế, cần có những nghiên cứu cải tiến thêm như:

Thiết kế lại giải thuật, trang bị thiết bị, cung cấp hạ tầng để có thể vận hành được với số lượng camera lớn.

Nâng cấp giải thuật nhận dạng kết hợp với tự động thêm dữ liệu huấn luyện bổ sung để xác định được người lạ xuất hiện bao nhiêu lần trong quá trình giám sát an ninh của hệ thống và thời điểm xuất hiện của người lạ được lưu vết có thể trích lục, thống kê.

Xây dựng giao diện đồ họa hoàn chỉnh và hoàn thiện thêm các tính năng để thuận tiện cho người dùng trong khai thác.

TÀI LIỆU THAM KHẢO

- [1] P. Viola and M. Jones, "Robust real time face detection ,international journal of computer vision," 57, 2004.
- [2] P. ; Viola and M. Jones, "Rapid Object Detection Using a Boosted Cascade of Simple Features," 2004. [Online]. Available: <http://www.merl.com>
- [3] Yang, B.; Yan, J.; Lei, Z.; and Li, S. Z. 2014. Aggregate channel features for multi-view face detection
- [4] Pham, M. T.; Gao, Y.; Hoang, V. D. D.; and Cham, T. J. 2010. Fast polygonal integration and its application in extending haar-like features to improve object detection. In Computer Vision and Pattern Recognition
- [5] LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). "Backpropagation Applied to Handwritten Zip Code Recognition." Neural Computation, 1(4), 541-551
- [6] Zhang, K., Zhang, Z., Li, Z., & Qiao, Y. (2016). "Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks." IEEE Signal Processing Letters, 23(10), 1499-1503
- [7] Schroff, F., Kalenichenko, D., & Philbin, J. (2015). "FaceNet: A Unified Embedding for Face Recognition and Clustering." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 815-823
- [8] Jin, R., Li, H., Pan, J., Ma, W., & Lin, J. (2021). Face Recognition Based on MTCNN and FaceNet. www.aaai.org

